

NE 511: Final Report

Joseph F. Specht IV

April 30, 2026

Anderson acceleration for PWR whole-core pin-by-pin Nu-TH coupling calculation in NECP-Bamboo2.0

1 Multi-Physics Phenomenon

In this paper, the researcher from Xi'an Jiaotong University and his colleague are solving high-fidelity, pin-by-pin, neutronics/thermal-hydraulics coupled, multiphysics on an whole-core pressurized water reactor (PWR) model. The team implemented Anderson Acceleration (AA) in their in-house code NECP-Bamboo2.0, where NECP is the Nuclear Engineering Computational Physics group at Xi'an Jiaotong University. All neutronics calculations take place within the NECP-Bamboo2.0 framework while the thermal-hydraulics calculations are solved using a two-phase, homogeneous-flow, subchannel solver called START.

1.1 Neutronics

In traditional, high-fidelity, reactor physics, assemblies are usually homogenized to find the assembly-averaged cross-sections and the desired quantities. In the Bamboo2.0 framework, the homogenization occurs on a pin-by-pin level, so the resulting solution is much higher fidelity. The "Bamboo-Lattice" module is responsible for calculating the pin-by-pin cross sections. For each unique combination of fuel pin and surrounding water, Bamboo-lattice solves the 2D neutron transport equation with 69 energy groups. Then, the code performs energy condensation where the 69 groups are condensed down to 4 or 7 energy groups. The results of these calculations are then stored in a lookup table that accounts for variables like burn-up, fuel temperature, and moderator density.

After calculating the required cross-sections, the cross sections are fed into the Bamboo-Core module. The neutronics model used in Bamboo-Core is based on an SP_3 solver that uses the Exponential Functional Expansion Nodal (EFEN). The SP_3 method is the simplified diffusion method using third order Legendre

polynomials. The SP_3 method provides angular feedback, unlike the diffusion method, without requiring the computational resources required when solving the full transport equation, like the S_n method.

1.2 Thermal-Hydraulics

The thermal-hydraulics (TH) calculations are performed by the Stead and Transient Analysis for Reactor Thermal-hydraulics (START) code. START is a two-phase, homogeneous-flow, subchannel solver developed by the same NECP lab who developed Bamboo2.0. START both calculates the temperatures and densities of the coolant and handles all of the heat transfer in the multiphysics solution.

1.3 Coupling

Due to the tightly coupled nature of Neutronics and Thermal-Hydraulics (Nu-TH) in real life, modeling the feedback between the two physics is critical for accurate calculations. The coupling is two-way such that the solution generated by the Nu code is used by the TH code to find the temperature and density profiles, which are then sent back to the Nu code. The previous cycle repeats until each solution stabilizes. However, when solving pin-by-pin models, the traditional coupling method of Picard iteration is too slow and prone to instabilities. The high-fidelity nature of the problem requires a method more stable than Picard iteration because the increased number of Degrees of Freedom (DOFs) give rise to an inherently less constrained and more instability-prone system.

2 Computational Methods

2.1 Typical Nu-TH

This subsection explains the various computational methods employed in traditional Nu-TH coupling methods. To understand how Anderson Acceleration differs from typical coupling methods, we should have a preliminary understanding of the computational methods employed in a typical Nu-TH simulation.

Classical Operator-Splitting is the simplest multiphysics coupling method. In classical operator-splitting, the neutronics and the thermal-hydraulics codes are run sequentially once per time step and the data are usually transferred externally. Operator-splitting is limiting if both codes converge on dramatically different time scales – different convergence rates means the global convergence rate will be determined by the slower converging code. Usually, operator-splitting only involves a single transfer of data from the parent physics code to the child physics code.

Jacobian-Free Newton-Krylov (JFNK) directly obtains a solution to the various physics fields based on Newton's method. The JFNK method precludes the construction of a full Jacobian matrix when solving the multiphysics equations. To solve the problem, we need to invert the Jacobian – the direct inversion cost of a matrix scales like M^3 , where M is the number degrees of freedom (DOFs), which quickly becomes computationally intractable for problems with many DOFs. The JFNK method can provide a substantial performance increase because we no longer have to invert the Jacobian. However, implementing the JFNK method requires extensive reworking of existing codes that utilize operator-splitting/Picard iteration. Therefore, many existing pin-by-pin codes (e.g. SCOPE2, DYN3D, PARCS) forgo implementation of the JFNK method in favor of Picard iteration for Nu-TH coupling.

Picard Iteration Picard iteration improves the classical operator-splitting approach by iterating at each time step until each physics field converged. For a multiphysics problem with two physics fields, Picard iteration involves (i) solving the first physics field, (ii) using the solution of the first field as the input for the second physics field, (iii) solving the second physics field, (iv) using the solution of the second physics field as the input for a new solution of the first physics field, (v) comparing the old and new solutions for the first physics field to see if the solution has converged, and (vi) continuing the iteration if the solution has not converged. Picard iteration can involve more than two physics fields and the fields can be arranged so with multiple levels of coupling as well.

Although Picard iteration is an improvement over classical operator-splitting, there are still disadvantages. One common disadvantage is known as "local over-convergence." In a multiphysics solution, the different physics fields might not converge at the same rate. For example, say we are solving for neutron flux and temperature. If we solve the neutron flux out to five decimal places, we are wasting computational resources if the temperature distribution is only accurate to one decimal place. One improvement people have made to increase the stability of Picard iteration is the use of relaxation factors. Relaxation factors use a portion of the the previous solution with the new solution instead of using the new solution directly. Relaxation factors slow convergence by not updating the solution as fast, but build a sort of "inertia" into the problem that increases stability.

2.2 Anderson Iteration/Acceleration

Anderson acceleration (AA) is an improvement to Picard iteration that serves as an alternative to relaxation factors. Instead of using only the previous iteration step when representing the new solution, AA expresses the current solution as a linear combination of previous iterations. According to theoretical analysis, AA has been shown to be equivalent to the generalized minimal residual method (GMRES). GMRES is a Krylov

subspace method that solves a linear system by minimizing the residual of the system given by $\|b - Ax_k\|$. GMRES generates a subspace of vectors and represents the solution as a linear combination of vectors in the subspace. However, GMRES is usually used to solve a linear system for a single physics field, whereas AA is used to solve a non-linear system of coupled physics fields. Aside from the level of application of the technique, AA and GMRES are essentially the same method.

AA is a highly attractive method for various reasons. First and foremost, AA was chosen for implementation in NECP-Bamboo2.0 because the method is effectively "plug and play" into current solvers because it is a modification to Picard iteration. AA is also applied at the global coupling level, so no changes have to be made to any physics codes. Second, AA converges faster than Picard iteration because it uses the history of previous solutions and residuals to extrapolate the solution. Therefore, AA can skip over steps where the residual will not change much and only calculate at the most important iterates. Third, AA is self-stabilizing as it finds the weighted average of past solutions that minimizes the residual, which results in less overshooting that is common in Picard iteration. Finally, the more DOFs, the more efficient the method becomes compared to traditional, Jacobian based solvers. For a pin-by-pin, full-core calculation, there are tens of thousands of fuel pins each with hundreds of DOFs. Instead of inverting a Jacobian $\in \mathbb{R}^{\#DOFs \times \#DOFs}$, AA inverts a matrix $\in \mathbb{R}^{m \times m}$, where m is the number of previous solutions being used to construct the current solution.

3 Computational Tools

3.1 NECP-Bamboo2.0

The neutronics capabilities in NECP-Bamboo2.0, thermal-hydraulics capabilities in START, and the coupling between the two were previously detailed in §1. However, NECP-Bamboo2.0 is a full-cycle simulation tool that also calculates the reactor's chemistry, control systems, and depletion. The remaining capabilities of NECP-Bamboo2.0 will be detailed in this subsection.

3.1.1 Criticality Search

In a real reactor, we modulate the reactivity via control mechanisms, such as: moving control rods and injecting borated water. During steady state operation, the reactor is critical such that the multiplication factor equal one, $k_{eff} = 1$. Therefore, NECP-Bamboo2.0 includes two methods of criticality search to find a critical reactor configuration. The first method is the soluble boron search. In real life, reactor operators can adjust the boron concentration in response to reactor feedback to maintain criticality. Similarly,

NECP-Bamboo2.0 iteratively adjusts the boron concentration in the coolant to solve for the exact boron concentration where the reactor is critical. Additionally, NECP-Bamboo2.0 is capable of a control rod position search for reactors without soluble boron. The code iteratively searches to find the exact insertion depth of the control rod to achieve criticality. However, the control rod position search was only mentioned as a capability of the code and not discussed in depth.

3.1.2 Micro-Depletion

Traditional codes use macro-depletion based on a global burnup number. However, NECP-Bamboo2.0 uses micro-depletion to track the concentrations of important nuclides in each fuel pin, which is far more accurate than macro-depletion. As the physics of the system does not change, the important nuclides micro-depletion are the same as the important nuclides in macro-depletion. NECP-Bamboo2.0 tracks nuclides including actinides (e.g. U-235, U-238, and Pu-239), neutron poisons (e.g. Xe-135, Sm-149), and precursors (e.g. Pm-149, I-135). The code solves the Bateman equations using the inbuilt point burnup solver, which uses the Chebyshev Rational Approximation Method (CRAM) or Transmutation Trajectory Analysis (TTA) method.

3.2 VERA Benchmarks

To ensure NECP-Bamboo2.0 solves the multi-physics correctly, the code is verified against the VERA Core Physics Benchmark Problems. These benchmarks are a series of problems based on the Watts Bar Nuclear 1 reaction, which was PWR-core designed by Westinghouse. VERA problem 6 is a 3D, single-assembly benchmark that validates the Nu-TH capabilities while VERA problem 7 is a 3D, whole-core benchmark that validates the Nu-Th/criticality/micro-depletion capabilities.

3.2.1 VERA Benchmark Problem 6

Problem 6 is a single, 17x17 fuel assembly with axial reflectors and two types of spacer grids. To solve this problem, the code needs to resolve the temperature distribution within the fuel pins and the density of water as the heats up. For this problem, the TH feedback effects are considered at zero-burnup, half-full-power conditions. The boron concentrations are given and the effect of fission products is omitted.

3.2.2 VERA Benchmark Problem 7

Problem 7 is a 3D, whole-core criticality search problem at hot full power. The core is based on a zero-burnup case with the micro-depletion equilibrium of Xe-135 taken into consideration.

4 Logical Path of Accomplishment

To implement Anderson Acceleration in NECP-Bamboo2.0, the researchers first had to identify the problem. In this case, the Picard iteration caused a bottleneck in the computations as the computational costs became intractable when solving pin-by-pin, coupled Nu-Th, multiphysics problems. Picard iteration converged too slowly and also oscillated around the true solution – these issues were only magnified when considering whole-core scenarios.

Once the problem was identified, the team decided to implement AA. They chose to implement AA as it did not require massive amounts of code restructuring. Additionally, the team had to consider the optimal memory depth, the number of previous solutions used to represent the next solution. If the memory depth was too deep, the solution would become unstable. If the memory depth was too shallow, the solver would not accelerate.

The researchers also incrementally increased the complexity of the models they tested their code on. In previous work, the NECP-Bamboo2.0 team tested their code on a single pin-cell to ensure the basic feedback loop was working. Then they tested it on a 2D assembly to verify radial feedback. Once the preliminary tests were completed, the team moved to verifying against more advanced benchmarks, which is where the scope of this paper begins.

For the advanced tests, the NECP-Bamboo2.0 team verified their code against the VERA benchmark problems 6 and 7. First, the team tested the code using VERA problem 6 to ensure AA could handle 3D axial feedback. Then, the team verified against VERA problem 7 to ensure the AA implementation could handle a pin-by-pin, whole-core, coupled Nu-TH solution that also included micro-depletion and criticality searches.

To demonstrate the success of their implementation, the team compared AA with the previous iteration scheme of Picard iteration. During testing, the team also ensured their solutions matched the high-fidelity reference solutions. Additionally, they recorded the wall-clock time to demonstrate the AA achieved the desired speed-up result.

5 Challenges and Their Solutions

During the implementation of AA, the NECP-Bamboo2.0 team overcame several challenges. The first problem was the strong feedback between the fuel temperature and water density. The strong feedback coupling was the original reason the NECP-Bamboo2.0 team implemented AA because the solution using Picard iteration oscillated too severely. To rectify this issue, the NECP-Bamboo2.0 team used a sort of safeguard

where the code reverts to a previous solution of the residual increases. The safeguard prevents the code from crashing even during hard transients like control rod movement.

The second problem the team encountered was the optimal memory depth. The current solution is represented as a linear combination of m previous iterations. If m was too shallow, AA would not provide the desired speed-up. If m was too deep, too many solutions would be store and the memory requirements would be too large. The team found the optimal value of m to be between 3 and 5.

Another challenge was the partial convergence interference between the Nu and TH solvers. Both solvers are individually iterative. Therefore, the team needed to synchronize the convergence criteria to ensure the Nu iterations were converged to a specific threshold before global iteration could occur. The convergence of the Nu solution was more important because the feedback is more sensitive than in the TH code.

6 Summary of Results

For VERA problem 6, the iteration performance of AA was compared against Picard iteration with and without a relaxation factor. Without relaxation factors, Picard iteration took 222 global iterations and 294 seconds to converge. Using a relaxation factor, the best trial took 12 global iterations and 13.6 seconds. However, both Nu-AA and TH-AA outperformed the Picard iteration regardless of relaxation factor. The best Nu-AA trial used an m of 2 and took 11.8 seconds while the best TH-AA trial used an m of 2 and took 11.3 seconds. For VERA problem 6, AA was shown to be better than Picard iteration with or without relaxation.

Additionally, the results of NECP-Bamboo2.0 for problem 6 was compared against the reference MPACT/CTF solution. Additionally, the team included MC21/CTF and MC21/COBRA-1E trials as references to similar codes. Out of the three codes being compared to MPACT/CTF, NECP-Bamboo2.0 was the closest to the reference eigenvalue with a difference of +24pcm compared to the reference solution while MC21/CTF and MC21/COBRA-1E both overestimated the eigenvalue by +63pcm. Additionally, the RMS of pin-power distribution bias was only 0.26% and the max pin-power distribution bias was -0.48%. The NECP-Bamboo2.0 results showed excellent agreement with the reference MCPACT/CTF solution.

For VERA problem 7, Picard iteration with and without relaxation, Nu-AA, and TH-AA were compared. Picard iteration without relaxation reached the maximum number of transport fission source iterations and diverged. The optimal relaxation trial converged in 14 global iteration in 176.9 seconds. The best Nu-AA trial with $m=8$ converged in 8 global iterations and finished in 111.4 seconds. The best TH-AA trial with $m=4$ converged in 23 iterations in 224.1 seconds. For problem 7, Nu-AA performed better than Picard iteration with or without relaxation, but TH-AA did not.

When compared against the MPACT/CTF results for problem 7, NECP-Bamboo2.0 predicted a critical boron concentration 4 pcm less than the reference solution. Additionally, the RMS of assembly power distribution bias was 0.37% while the maximum assembly power distribution bias was 0.69%. Once again, NECP-Bamboo2.0 showed excellent agreement with the reference solution comparable to the other codes under comparison.

Overall, it was shown the implementation of AA in NECP-Bamboo2.0 resulted in substantial performance increased via wall-clock time reduction. Additionally, NECP-Bamboo2.0 with AA showed excellent agreement with the reference solution from MPACT/CTF for VERA benchmarks problem 6 and 7.